

FlexInfer: Flexible LLM Inference Using CPU Computation

[Seonjin Na](#)¹, Geonhwa Jeong², Byung Hoon Ahn³, Aaron Jezghani¹,
Jeffery Young¹, Christopher J. Hughes⁴, Tushar Krishna¹, Hyesoon Kim¹

¹Georgia Institute of Technology, ²Meta, ³University of California San Diego, ⁴Intel Labs



Meta

UC San Diego

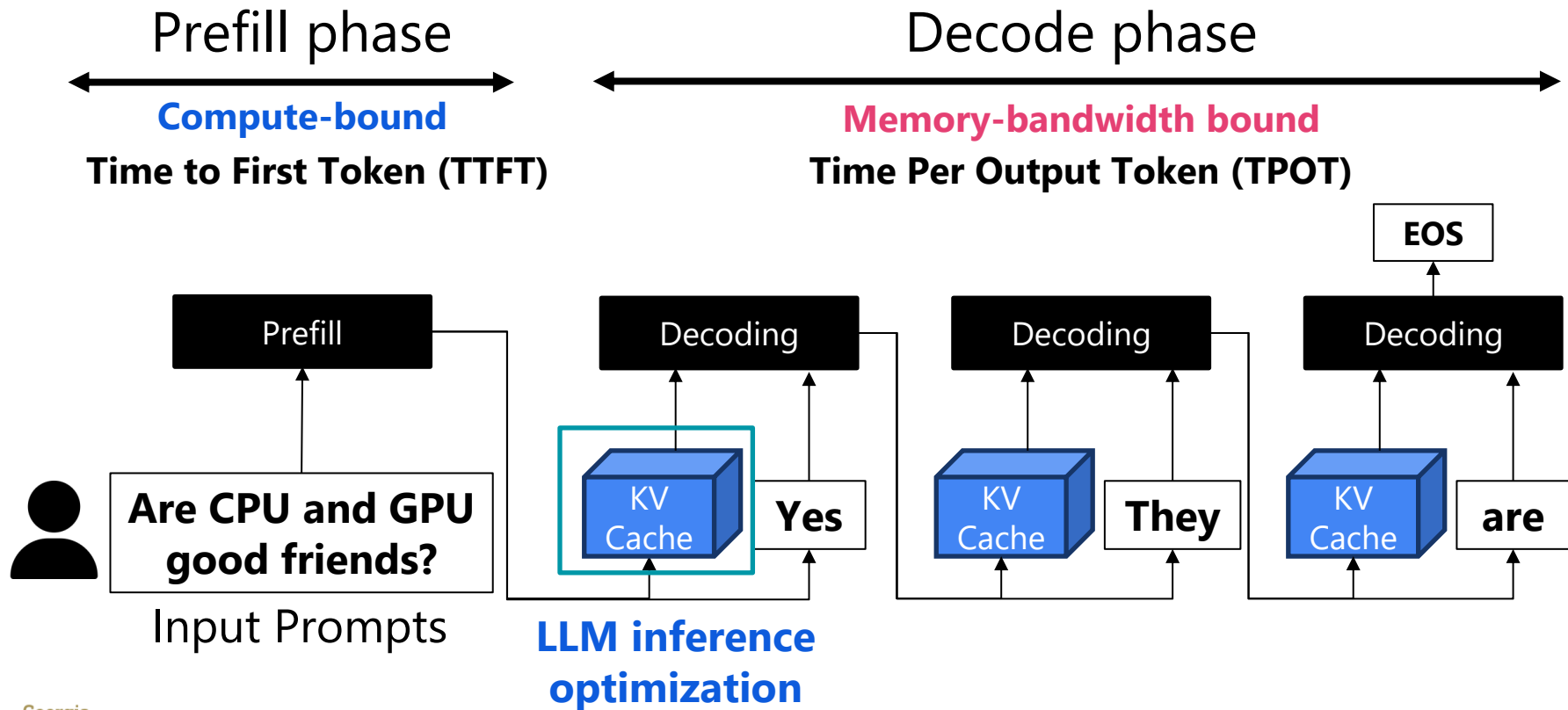


Increasing Demands of Large Language Models (LLM)

Data centers equip with
GPUs, NPUs to accelerate LLM inference

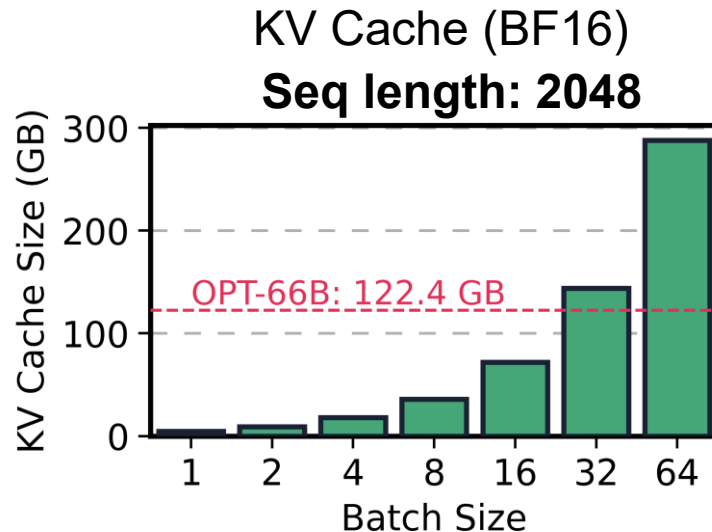
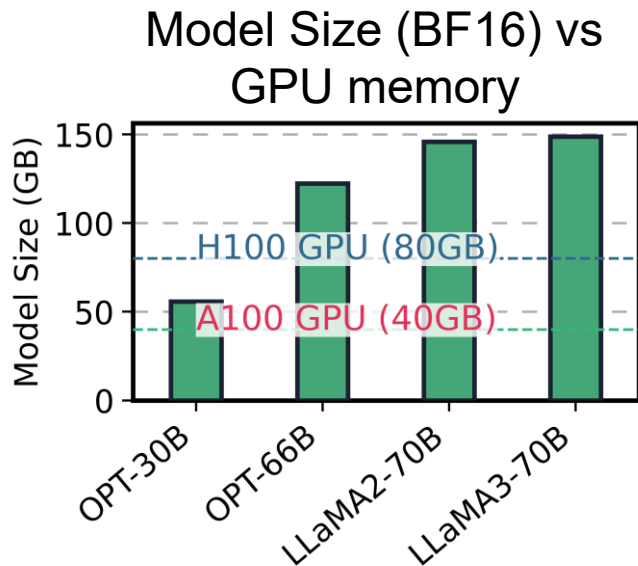


Overview of LLM Inference



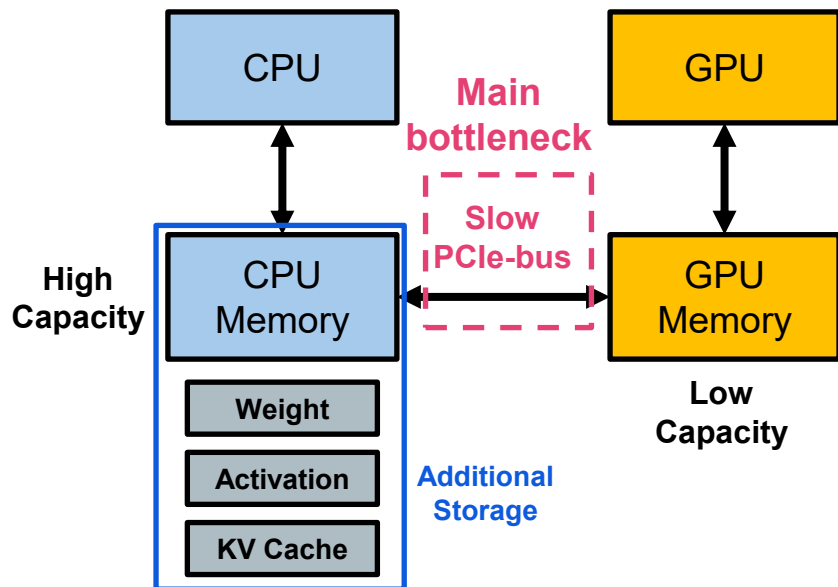
Memory Capacity Challenges in LLM Inference

- LLM inference requires **substantial amount of memory**
 - **The size of LLMs and KV cache**

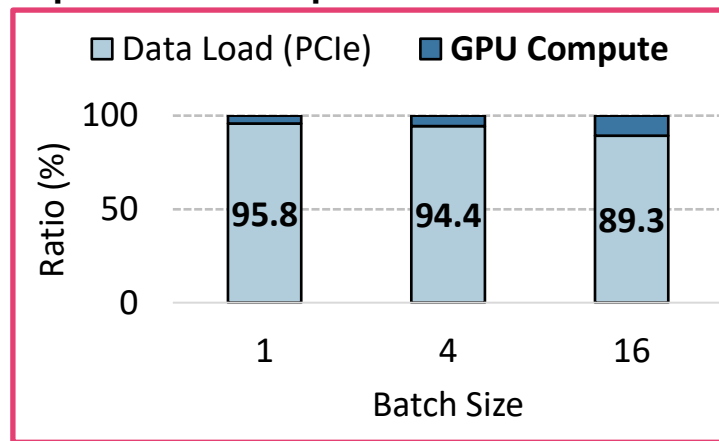


Offloading-based LLM Inference on GPUs

- Model weights, activations, KV cache are offloaded to CPU memory
 - HuggingFace Accelerate[1], DeepSpeed-ZeRO[2], FlexGen[3]



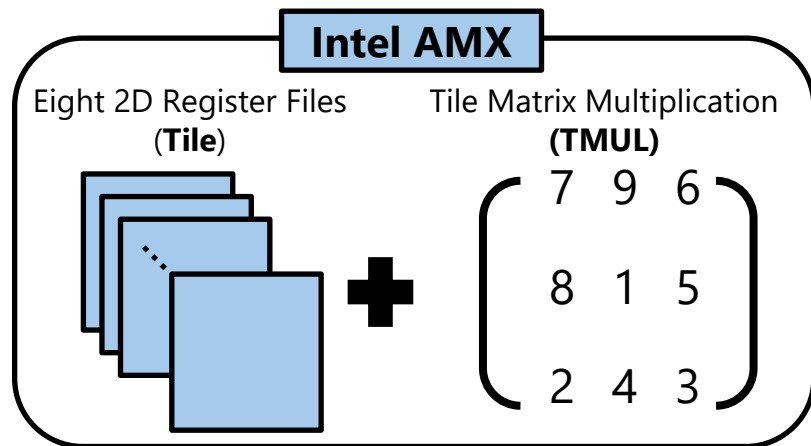
LLaMA2-70B
Input: 1024 / Output:32 on H100 FlexGen



> 89% execution time is spent on PCIe data transfer

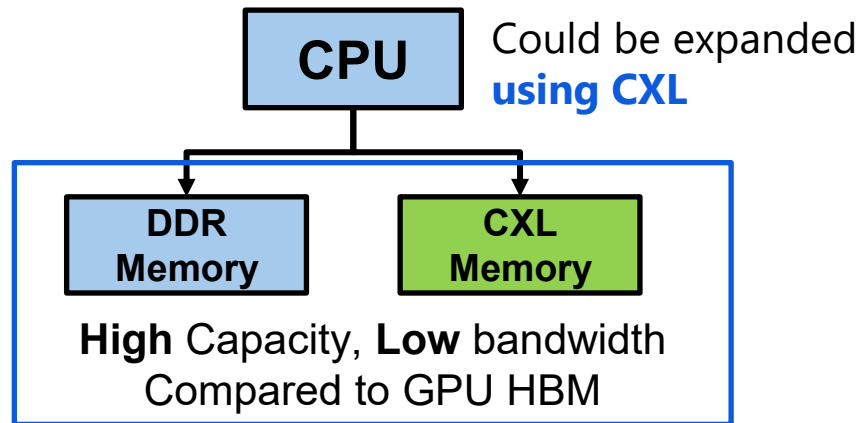
Key Opportunities in Recent CPUs

Dedicated GEMM Accelerator



Supported data type:
BF16, INT8

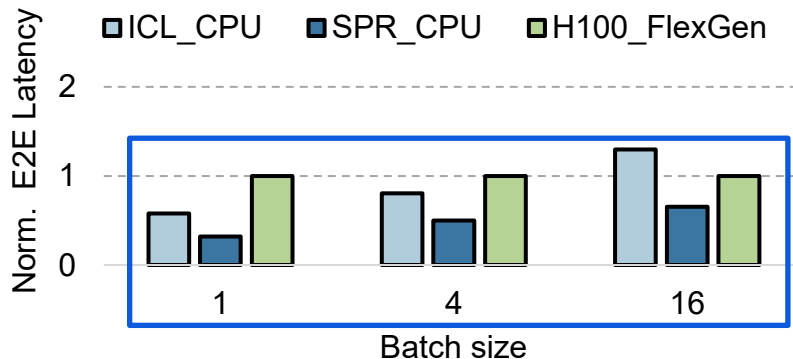
Larger Memory Capacity



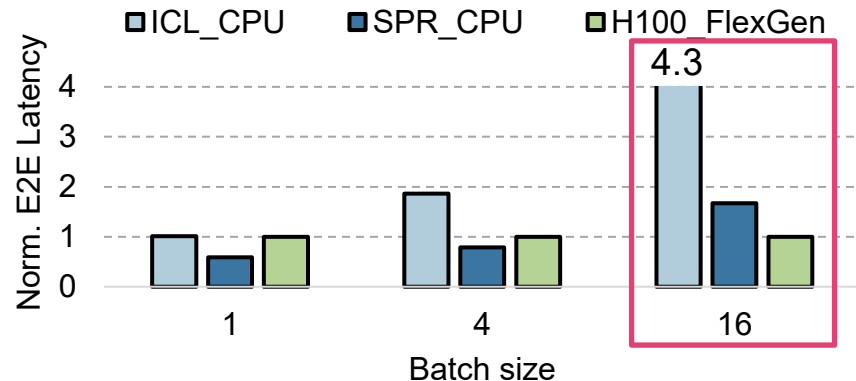
Motivation: CPU Compute vs GPU with Offloading

- End-to-end latency = Prefill (TTFT) + decode (TPOT * # tokens)
 - ICL CPU: CPU **without on-chip accelerator** + DDR4
 - SPR CPU: CPU **with on-chip accelerator** + DDR5
 - H100 FlexGen: Weights, KV cache in CPU memory

Input length: 128 output length: 32



Input length: 1024 output length: 32



Motivation: CPU Compute vs GPU with Offloading

- TTFT vs TPOT breakdown results

- ICL CPU: CPU **without on-chip accelerator** + DDR4

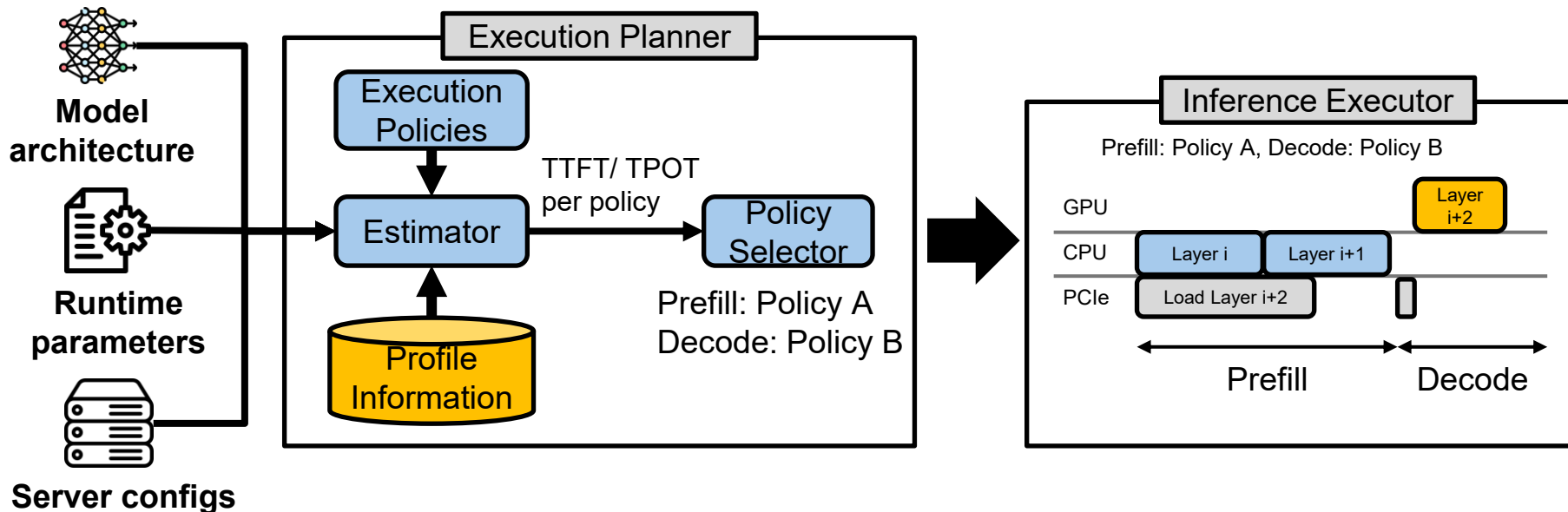
- SPB CPU: CPU **with on-chip accelerator** + DDR5

1. **CPU Compute: good for decode, but limited throughput for prefill with longer input lengths and larger batch sizes**
2. **Adaptively leveraging CPU computation is crucial for optimal performance**

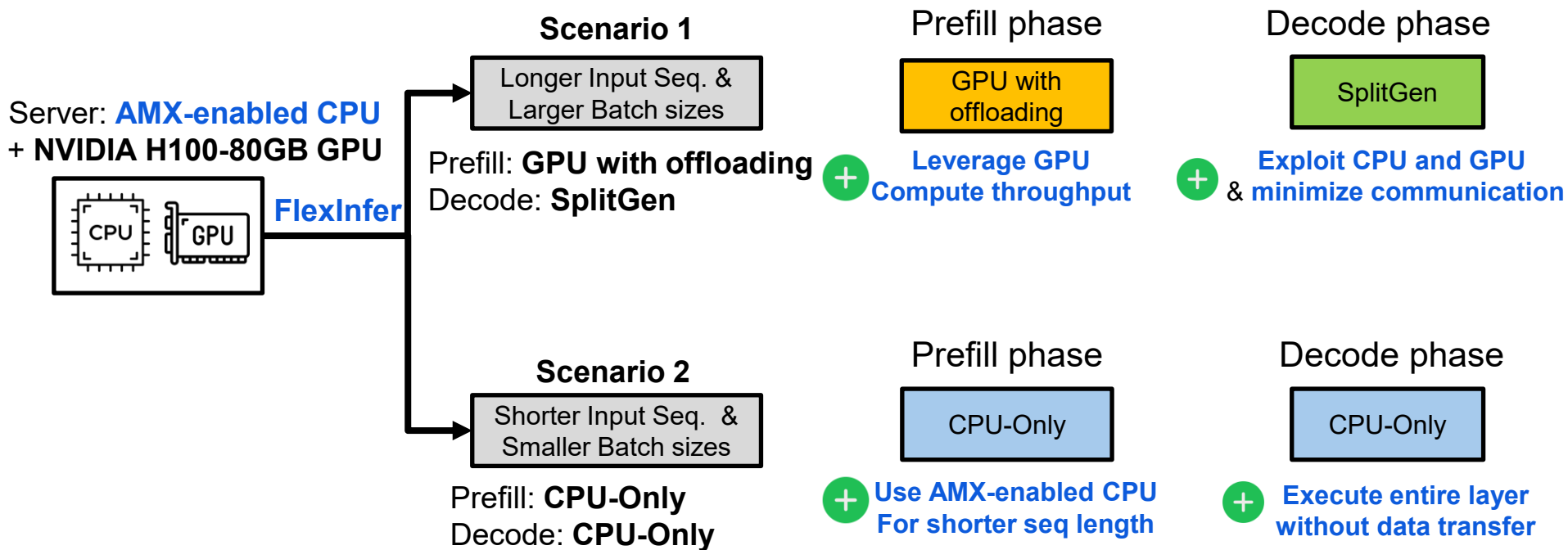


Overview of FlexInfer

- Goal: **dynamically select execution policies** to take advantages of CPU and GPU
 - Based on runtime parameters, model architecture, and hardware configurations



Dynamic Policy Selection Examples By FlexInfer



Evaluation Methodology

- **Baselines**

- CPU-only (ICL, SPR CPUs)
- FlexGen
- FlexGen_Opt
- SplitGen

- **Models**

- OPT-30B, 66B
- LLaMA2-70B, LLaMA3-70B

Non-AMX CPU
+ A100-40GB GPU

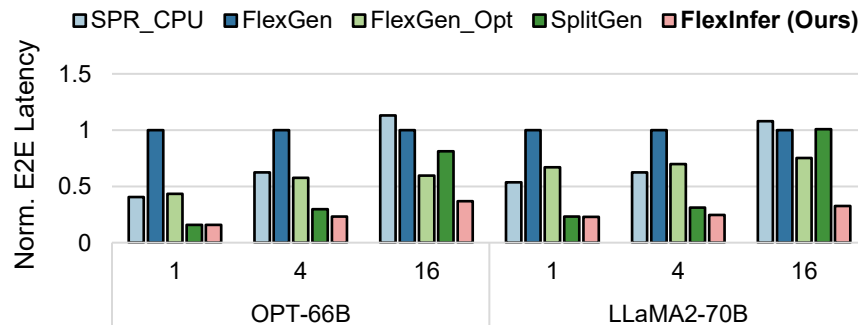
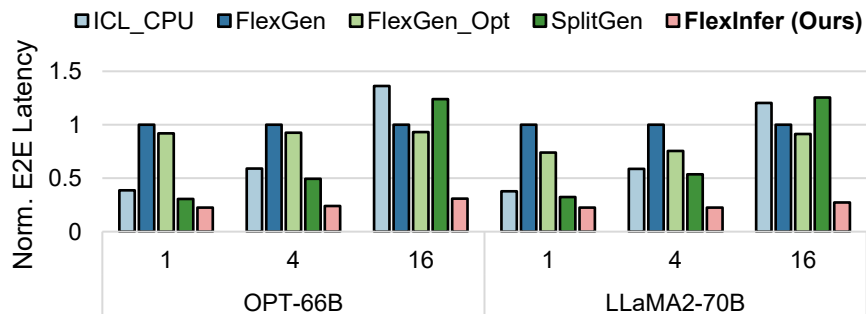
AMX-enabled CPU
+ H100-80GB GPU

	Server 1	Server 2
CPU	2x ICL CPU, 256GB	2x SPR CPU, 512GB
GPU	A100 40GB GPU	H100 80GB GPU
Compute Throughput (BF16)	CPU: 9 TFLOPS GPU: 312 TFLOPS	CPU: 144.2 TFLOPS GPU: 756 TFLOPS
Memory Bandwidth	CPU: 311.8 GB/s GPU: 1299.9 GB/s	CPU: 407.8 GB/s GPU: 1754.4 GB/s
CPU-GPU Interconnect	PCIe 4.0, 32 GB/s	PCIe 5.0, 64 GB/s

Memory bandwidth is measured using STREAM benchmark

Evaluation Results: End-to-End Latency Comparison

- Runtime parameters: Input length 512, output length 32
- **FlexInfer** reduce inference latency by **75%, 76%** on average for each server



More Details in Our Paper

- Performance estimator
- Sensitivity study results
 - Performance impact of varying input lengths
 - Performance impact of varying output lengths
- PCIe traffic analysis
- Others ...

Conclusion

- **Problem**

- LLM inference requires **substantial amount of memory**, often exceeding recent GPU memory
- Offloading-based LLM inference suffer from performance degradation **due to data transfer**

- **Key Idea**

- CPU computation has potential but still limited to process prefill phase with longer seqs
- **FlexInfer** dynamically select execution strategies for prefill and decode phase
 - Based on hardware configurations, runtime parameters

- **Evaluation results**

- **FlexInfer** significantly reduces end-to-end latency by **75%, 76%** on two different servers